

tf.TypeSpec Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/TypeSpec

Specifies a TensorFlow value type.

Code Examples

```
spec = tf.TensorSpec(shape=[None, None], dtype=tf.int32)
@tf.function(input_signature=[spec])
def double(x):
    return x * 2
double(tf.constant([[1, 2], [3, 4]]))
```

```
spec = tf.TensorSpec(shape=[None, None], dtype=tf.int32)
```

```
@tf.function(input_signature=[spec])
```

```
def double(x):
```

```
    return x * 2
```

```
double(tf.constant([[1, 2], [3, 4]]))
```

```
numpy=array([[2, 4], [6, 8]], dtype=int32)>
```

Documentation

Specifies a TensorFlow value type.

Inherits From: [TraceType](#)

View aliases

Compat aliases for migration

See

[Migration guide for more details.](#)

[tf.compat.v1.TypeSpec](#), [tf.compat.v1.data.experimental.Structure](#)

A `tf.TypeSpec` provides metadata describing an object accepted or returned by TensorFlow APIs. Concrete subclasses, such as `tf.TensorSpec` and `tf.RaggedTensorSpec`, are used to describe different value types.

For example, `tf.function`'s `input_signature` argument accepts a list (or nested structure) of `TypeSpecs`.

Creating new subclasses of `TypeSpec` (outside of TensorFlow core) is not currently supported. In particular, we may make breaking changes to the private methods and properties defined by this base class.

Example:

`## Attributes`

In particular, all values that are compatible with this `TypeSpec` must be an instance of this type.

Methods

`## experimental_as_proto`

[View source](#)

Returns a proto representation of the `TypeSpec` instance.

Do NOT override for custom non-TF types.

`experimental_from_proto`

[View source](#)

Returns a `TypeSpec` instance based on the serialized proto.

Do NOT override for custom non-TF types.

`experimental_type_proto`

[View source](#)

Returns the type of proto associated with TypeSpec serialization.

Do NOT override for custom non-TF types.

is_compatible_with

[View source](#)

Returns true if spec_or_value is compatible with this TypeSpec.

Prefer using "is_subtype_of" and "most_specific_common_supertype" wherever possible.

is_subtype_of

[View source](#)

Returns True if self is a subtype of other.

Implements the tf.types.experimental.func.TraceType interface.

If not overridden by a subclass, the default behavior is to assume the TypeSpec is covariant upon attributes that implement TraceType and invariant upon rest of the attributes as well as the structure and type of the TypeSpec.

most_specific_common_supertype

[View source](#)

Returns the most specific supertype TypeSpec of self and others.

Implements the tf.types.experimental.func.TraceType interface.

If not overridden by a subclass, the default behavior is to assume the `TypeSpec` is covariant upon attributes that implement `TraceType` and invariant upon rest of the attributes as well as the structure and type of the `TypeSpec`.

`most_specific_compatible_type`

[View source](#)

Returns the most specific `TypeSpec` compatible with self and other. (deprecated)

Deprecated. Please use `most_specific_common_supertype` instead.
Do not override this function.

`__eq__`

[View source](#)

Return `self==value`.

`__ne__`

[View source](#)

Return `self!=value`.